

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 12
“SEARCHING”



DISUSUN OLEH:
RAFI RAMADHAN
109082500140
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Searching atau pencarian data adalah proses menemukan suatu nilai atau elemen tertentu dalam sekumpulan data. Berbeda dengan pencarian nilai ekstrim seperti maximum atau minimum yang selalu berhasil menemukan nilai, pencarian data spesifik memiliki kemungkinan bahwa data yang dicari tidak ditemukan dalam kumpulan data tersebut. Secara formal, diberikan array $T[0..N-1]$ dan nilai X yang ingin dicari, algoritma searching akan mengembalikan status ditemukan berupa boolean (true/false) atau mengembalikan indeks posisi data — nilai lebih dari atau sama dengan 0 jika ditemukan, dan -1 jika tidak ditemukan.

Sequential Search adalah algoritma pencarian yang memeriksa setiap elemen array satu per satu secara berurutan mulai dari elemen pertama hingga elemen terakhir, atau berhenti begitu data ditemukan. Karena tidak membutuhkan data yang terurut, algoritma ini dapat diterapkan pada semua kondisi array dan juga pada pencarian berdasarkan field apa saja dalam array bertipe struct. Kompleksitas waktunya adalah $O(1)$ pada kasus terbaik ketika data berada di elemen pertama, dan $O(n)$ pada kasus terburuk ketika data berada di akhir atau tidak ada sama sekali. Selain mengembalikan nilai boolean, Sequential Search juga dapat dimodifikasi untuk mengembalikan indeks posisi data dengan nilai -1 sebagai penanda data tidak ditemukan.

Binary Search adalah algoritma pencarian yang jauh lebih efisien dengan kompleksitas $O(\log n)$, namun mensyaratkan data dalam keadaan terurut baik secara ascending maupun descending. Prinsip kerjanya adalah membagi rentang pencarian menjadi dua bagian secara berulang — ambil elemen di posisi tengah antara batas kiri dan batas kanan, lalu bandingkan dengan nilai yang dicari. Jika elemen tengah terlalu kecil maka rentang digeser ke kanan, jika terlalu besar digeser ke kiri, dan jika sama berarti data ditemukan. Perlu diperhatikan bahwa logika Binary Search untuk data ascending dan descending berbeda dan tidak dapat saling dipertukarkan, serta khusus pada array bertipe struct, keterurutan array harus berdasarkan field yang sama dengan kunci pencarian yang digunakan.

SOAL LATIHAN

Statement perulangan

1. Pemilihan Ketua RT

Source Code:

```
package main

import "fmt"

const MAXCALON = 20

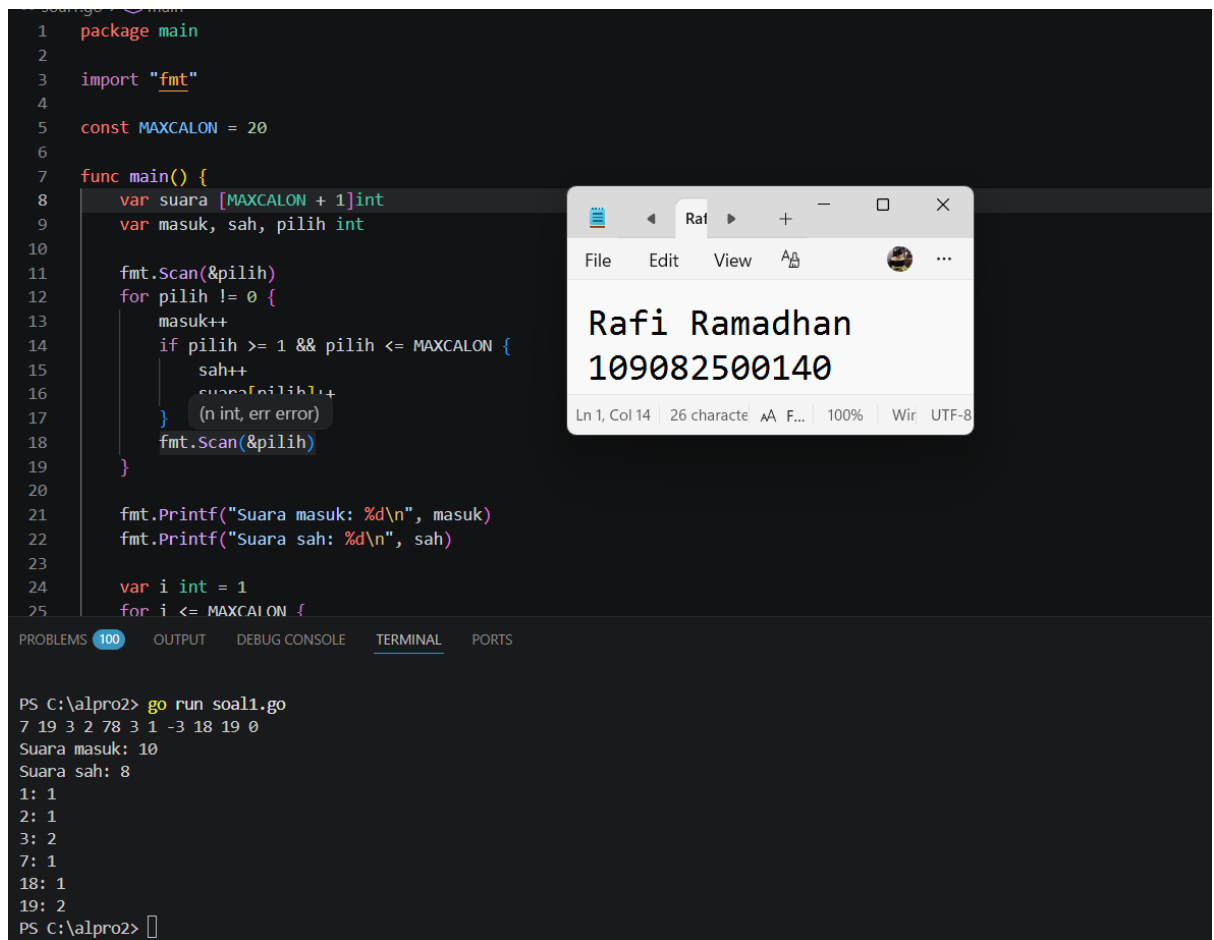
func main() {
    var suara [MAXCALON + 1]int
    var masuk, sah, pilih int

    fmt.Scan(&pilih)
    for pilih != 0 {
        masuk++
        if pilih >= 1 && pilih <= MAXCALON {
            sah++
            suara[pilih]++
        }
        fmt.Scan(&pilih)
    }

    fmt.Printf("Suara masuk: %d\n", masuk)
    fmt.Printf("Suara sah: %d\n", sah)

    var i int = 1
    for i <= MAXCALON {
        if suara[i] > 0 {
            fmt.Printf("%d: %d\n", i, suara[i])
        }
        i++
    }
}
```

Output:



The screenshot shows a Go IDE with a source code editor on the left and a terminal window on the right. The source code is a Go program for a voting system. It defines a constant `MAXCALON = 20` and a `main` function. The `main` function uses `fmt.Scan` to read a choice (`pilih`) and increments counters for valid choices (`masuk` and `sah`). It also prints the total counts. The terminal window shows the execution of the program with the input `7 19 3 2 78 3 1 -3 18 19 0` and the output: `Suara masuk: 10` and `Suara sah: 8`.

```
1 package main
2
3 import "fmt"
4
5 const MAXCALON = 20
6
7 func main() {
8     var suara [MAXCALON + 1]int
9     var masuk, sah, pilih int
10
11     fmt.Scan(&pilih)
12     for pilih != 0 {
13         masuk++
14         if pilih >= 1 && pilih <= MAXCALON {
15             sah++
16             suara[pilih]++
17         } (n int, err error)
18         fmt.Scan(&pilih)
19     }
20
21     fmt.Printf("Suara masuk: %d\n", masuk)
22     fmt.Printf("Suara sah: %d\n", sah)
23
24     var i int = 1
25     for i <= MAXCALON {
```

```
PS C:\alpro2> go run soal1.go
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS C:\alpro2>
```

Deskripsi Program:

Program membaca serangkaian bilangan bulat dari input yang mewakili suara warga dalam pemilihan ketua RT dengan 20 calon. Setiap bilangan yang masuk dicatat sebagai suara masuk, lalu divalidasi — hanya nilai 1 sampai 20 yang dianggap sah. Program berhenti membaca ketika menemukan nilai 0, lalu menampilkan total suara masuk, total suara sah, dan daftar calon yang mendapat suara beserta jumlahnya.

2. Pemilihan Ketua RT (Cari Pemenang Dan Wakil)

Source Code:

```
package main

import "fmt"

const MAXCALON = 20

func cariPemenang(suara [MAXCALON + 1]int) (int, int) {
    var ketua, wakil int = -1, -1
    var i int = 1
    for i <= MAXCALON {
        if suara[i] > 0 {
            if ketua == -1 || suara[i] > suara[ketua] {
                wakil = ketua
                ketua = i
            } else if wakil == -1 || suara[i] > suara[wakil] {
```

```

        wakil = i
    }
}
i++
}
return ketua, wakil
}

func main() {
    var suara [MAXCALON + 1]int
    var masuk, sah, pilih int

    fmt.Scan(&pilih)
    for pilih != 0 {
        masuk++
        if pilih >= 1 && pilih <= MAXCALON {
            sah++
            suara[pilih]++
        }
        fmt.Scan(&pilih)
    }

    fmt.Printf("Suara masuk: %d\n", masuk)
    fmt.Printf("Suara sah: %d\n", sah)

    ketua, wakil := cariPemenang(suara)
    fmt.Printf("Ketua RT: %d\n", ketua)
    fmt.Printf("Wakil ketua: %d\n", wakil)
}

```

Output:

```
soal2.go > cariPemenang
1 package main
2
3 import "fmt"
4
5 const MAXCALON = 20
6
7 func cariPemenang(suara [MAXCALON + 1]int) (int, int) {
8     var ketua, wakil int = -1, -1
9     var i int = 1
10    for i <= MAXCALON {
11        if suara[i] > 0 {
12            if ketua == -1 || suara[i] > suara[ketua] {
13                wakil = ketua
14                ketua = i
15            } else if wakil == -1 || suara[i] > suara[wakil] {
16                wakil = i
17            }
18        }
19        i++
20    }
21    return ketua, wakil
22 }
23
24 func main() {
25     var suara [MAXCALON + 1]int
26
27     PS C:\alpro2> go run soal2.go
28 7 19 3 2 78 3 1 -3 18 19 0
29 Suara masuk: 10
30 Suara sah: 8
31 Ketua RT: 3
32 Wakil ketua: 19
33 PS C:\alpro2>
```

Deskripsi Program:

Program ini adalah pengembangan dari Soal 1 dengan tambahan pencarian pemenang. Setelah menghitung suara, program mencari calon dengan suara terbanyak sebagai Ketua RT dan calon dengan suara terbanyak kedua sebagai Wakil Ketua RT. Apabila terdapat seri, calon dengan nomor peserta terkecil yang diprioritaskan, sehingga tidak diperlukan penanganan khusus karena loop pencarian sudah berjalan dari nomor 1 ke 20 secara berurutan.

3. Binary Search

Source Code:

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func isiArray(n int) {
    var i int = 0
    for i < n {
        fmt.Scan(&data[i])
        i++
    }
}
```

```

func posisi(n, k int) int {
    var found int = -1
    var kr int = 0
    var kn int = n - 1
    var med int

    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if k < data[med] {
            kn = med - 1
        } else if k > data[med] {
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func main() {
    var n, k int
    fmt.Scan(&n, &k)
    isiArray(n)

    hasil := posisi(n, k)
    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}

```

Output:

```
1 package main
2
3 import "fmt"
4
5 const NMAX = 1000000
6
7 var data [NMAX]int
8
9 func isiArray(n int) {
10     var i int = 0
11     for i < n {
12         fmt.Scan(&data[i])
13         i++
14     }
15 }
16
17 func posisi(n, k int) int {
18     var found int = -1
19     var kr int = 0
20     var kn int = n - 1
21     var med int
22
23     for kr <= kn && found == -1 {
24         med = (kr + kn) / 2
25         if k < data[med] {
```

Raf

File Edit View A 100% Wir UTF-8

Rafi Ramadhan

109082500140

Ln 1, Col 14 26 character F... 100% Wir UTF-8

```
106 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\alpro2> go run soal3.go
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\alpro2> go run soal3.go
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
PS C:\alpro2>
```

Deskripsi Program:

Program menerima n data integer positif yang sudah terurut membesar dan sebuah bilangan k yang ingin dicari. Menggunakan algoritma Binary Search, program mencari posisi k dalam array dan menampilkan indeksinya (dihitung dari 0) jika ditemukan, atau mencetak "TIDAK ADA" jika tidak. Binary Search dipilih karena n bisa mencapai 1.000.000, sehingga Sequential Search yang $O(n)$ tidak efisien — Binary Search menyelesaikannya hanya dalam sekitar 20 iterasi dengan kompleksitas $O(\log n)$.

DAFTAR PUSTAKA

Berikut adalah sumber-sumber referensi yang digunakan dalam penyusunan Modul 12 Searching:

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley Professional.

Tim Penyusun Modul. (2023). *Modul Praktikum Algoritma dan Pemrograman 2*. Fakultas Informatika, Telkom University.

Go Documentation Team. (2024). *The Go Programming Language Specification*. Google LLC. Diakses dari <https://go.dev/ref/spec>

Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching* (2nd ed.). Addison-Wesley.